

英国求职 + 面试作战手册 (Luohua Yang · Data Scientist / ML Engineer 应届)

基于当前简历 [Luohua_Yang_CV.pdf](#) 定制。目的：帮你尽快在英国落地一份 Data / AI 相关的 Graduate / Junior 岗。核心策略：海投 (量) + 精准定制 (质) + 面试弹药提前备好 (不临时抱佛脚)。

总览 (先读这个)

阶段	关键动作	目标
0. 简历收尾	填真实 Gmail / 手机号 / LinkedIn 链接	投递前 100% 可用
1. 准备齐	自我介绍背熟 + 项目讲解脱稿 + SQL/Python 题库刷一遍	面试不慌
2. 海投	每天 15~25 封，多渠道并行	1~2 周内拿到 3~5 次面试
3. 面试	STAR 讲故事 + 技术题过关 + 后面试环节准备提问	拿到 offer
4. 跟进	面试后 24h 内发 Thank-you email	提高转正率

心态：英国应届 Data 岗，投递量与面试机会的比例通常是 20:1 ~ 50:1。投 100~200 封拿 3~5 次面试是完全正常的数字，不是失败。所以量要够、心态要稳。

一、英国求职流程 (完整路径)

1.1 时间线 (你现在 8 月底，属于偏晚但可行)

英国 graduate scheme 大多 9~11 月开 (投次年 9 月入职)，但你走的是更高频的 "experienced hire / direct hire" 通道 (针对短期入职)，全年滚动招聘，这时候反而正合适，因为 graduate scheme 大批人挤，direct 岗位竞争小得多。

务实路线：别只盯着毕业生的正式校招 (graduate scheme，流程长、要 PS、OT、VI、AC 一轮轮)，优先投 direct hire 的 Junior/Entry-level Data/Business Intelligence/Analyst 岗，流程短、回国率快、你有真实项目能讲反而是加分项。

1.2 典型面试流程 (direct hire 通常 2~4 轮)

HR 电话 (15min, 筛选) → 技术电话/视频面试 (45-60min) → 线下/技术复试 or 团队面 (1h) → (可能) final → Offer

各轮考什么：

- **HR 初步**：自我介绍、motivation (为什么想做这个/为什么这个公司)、availability、现有身份 (是否需要 sponsorship，回答见下)、薪资预期
- **技术面**：SQL/Python 现场做题 + 简历项目深挖 + 1-2 个场景题 (design/思考逻辑)
- **团队面**：项目怎么做的、跟人怎么协作、一个过往挫折、行为题 (STAR)

1.3 ⚠️ 签证/身份问题 (必须想清楚)

这是英国求职最关键的现实约束，答不好直接刷掉：

- **你是什么身份？** 如果你是 **Graduate visa (毕业签证，2年)** → 不需要公司 sponsor，**这是你最大的谈判筹码**，一定要在 cover letter 开头讲清楚：*"I hold valid right to work in the UK for X years (Graduate visa), no sponsorship required."* 这能进 90% 不 sponsor 的中小公司。
- 如果只是想找工签 sponsor 的大公司 → 竞争激烈很多，要海投 + 早投。
- ⚠️ **你得先确认自己的签证状态**，这决定了投递范围。如果你有 Graduate visa，你的求职面会宽 10 倍。

🌐 二、投递网站 & 渠道 (按优先级)

2.1 主战场 (Python/Data/AI 岗位最集中)

平台	用途	特点
LinkedIn	第一投放渠道	每个岗位都能看，搜 "Data Analyst" "Junior Data Scientist" "Business Intelligence Analyst" "Junior Python Developer"，设 alert 每天推
Indeed UK (uk.indeed.com)	大量中小公司	量大，适合海投，直接投
Totaljobs / Reed	英国本地老牌	中小企业多，回复率高一点
Glassdoor	看薪资 + 面经	顺便投
Gradcracker	理工毕业生专项	偏 graduate scheme

2.2 Data/AI 专项 & 远程

平台	用途
Otta / Welcome to the Jungle	科技公司精准推送，UX 好，偏 startup
DataJobs / Eurotechjobs	Data 垂直站
Hired / Talent.io / Airswift (有 recruiter 帮你找)	技术人才平台
Y Combinator Work at a Startup	欧美 startup 集中，愿意办 visa 的也多

2.3 猎头 recruiters (容易被忽略但超有用)

在 LinkedIn 上 open profile，很多技术猎头会主动联系你。英国 tech recruiter 是求职的重要渠道，尤其你能免 sponsorship 时，猎头很喜欢推你。主动在本地多联系几个靠谱的。

2.4 公司官网直投

认出不错的公司 (尤其你俩 GitHub 项目相关的：金融、数据服务、SaaS、BI 类)，去 careers 页直投，绕过 ATS 更有机会被看到。

 三、投递技巧 & 执行策略

3.1 核心心法：投递给"人"，不给"系统"

- 每封简历尽量带 cover letter (求职信)，至少给第一梯队的目标公司写。
- Cover letter的黄金句式 (应届够用，不用长，4-5句即可)：
 - > "I'm a recent MSc Statistical Data Science graduate from the University of Birmingham, and I currently hold valid right to work in the UK (Graduate visa, no sponsorship required). I've been building end-to-end data systems — from automated pipelines and BI dashboards in Tableau to LLM-integrated analytics — and I'm excited to bring hands-on, real-project experience to [Company]'s [Role]."
 - 加一句跟公司/岗位的关联，证明你不是海投。

3.2 每天执行节奏 (可落地)

时段	动作	数量
早上 30min	LinkedIn/Indeed 用关键词搜当天新岗位，点开看，匹配就投	5-8封
下午 30min	graduate/猎头渠道 + 官网直投	5-8封
晚上随手	补前面漏掉的、回 recruiter 消息	3-5封
合计		15~20封/天

→ 一周 100+ 封，两周 200+ 封，足以产生 5~8 次面试池。宁可投快，不要精致地慢。

3.3 ⚠️ 别踩的坑

- ❌ 一份简历投所有岗不加改动 → 针对 **Data Analyst** 和 **Data Scientist/MLE** 微调关键词再投 (Data Analyst 强调 SQL/Tableau/BI ; MLE强调 ML/LLM) 。
- ❌ 只投大厂 → 应届技术岗 **中小公司+startup** 命中率远高于大厂。
- ❌ 投完就不管 → 面试后/投递后跟进。
- ❌ ATS 被卡 → 简历关键词要和 JD 一致 (说 SQL 就写 SQL , 别只写"数据库") 。

3.4 📧 跟进 (面试后 24h 内)

发封简短 Thank-you email :

"Thank you for your time today — I really enjoyed discussing your [X]. It reinforced my interest in [role], and I'm confident I can contribute to [specific thing discussed]. Happy to provide anything else. Looking forward to the next step."

👤 四、自我介绍 (两个版本 , 背熟)

核心结构 : 我是谁 → 我能做什么 → 我能给公司带来什么价值 → 为什么现在/为什么你

4.1 精简版 (HR 电话 / 30-45 秒)

"Hi, I'm Luohua Yang. I'm a recent MSc Statistical Data Science graduate from the University of Birmingham, with a background in Statistics from Guangdong University. What I love is owning the whole data journey — from scheduling the data collection, building pipelines, to visualising insights and even hooking in LLMs for automated reporting. I bring real, hands-on projects: an automated multi-market stock monitoring system that cut my manual research time by about 90%, and a news knowledge base that distills over 100 articles a day into a 5-minute briefing. In Python and SQL I'm very comfortable, and I've also built Tableau dashboards in an internship to drive content decisions. I'm looking for a data-focused role where I can build things end-to-end and grow quickly — and I'm excited to bring that project-driven mindset to your team."

4.2 详细版 (面试开场 / 1.5-2 分钟)

"Sure! Let me walk you through — who I am, what I build, and what I can bring. **Background:** I did my BSc in Statistics in China, then came to the UK for an MSc in Statistical Data Science at Birmingham. My thesis compared generative vs. discriminative classifiers for financial sentiment — so I have a solid ML foundation, not just tool usage. **What I do:** I build things end-to-end. Two highlights — 1) A **stock-monitor system**: automated daily quotes and news, analysed by a multi-provider LLM layer running Buffett/Fisher frameworks, all served on a Flask + ECharts dashboard with Feishu alerts. It cut manual research by about 90% and surfaces 5+ candidate opportunities daily. 2) A **news knowledge base**: a GitHub Actions pipeline aggregating four global sources, with a DeepSeek router that distills 100+ articles into a 5-minute bilingual briefing — massively improving my information intake. I also interned at Tungee, where I built the KPI pipeline for 50+ KOL accounts and Tableau dashboards that helped the operations team find top-performing content formats roughly twice as fast. **What I bring:** strong Python and SQL, real ML and LLM integration experience, and the ability to own a project from data to insight to delivery. I'm comfortable in both English and Chinese, and I work well both independently and on a team. **Why this role:** I want to apply this end-to-end, build-with-impact mindset to a real business — and I'm genuinely excited about [Company X]'s work in [领域]. That's me in a nutshell."

4.3 自我介绍的面试技巧

- 提前演练，控制在不会背稿感太重的程度；讲到项目处可以稍微放缓让人提问。
- 不要炫技堆术语，重点是"我做了、用什么、带来什么影响"。
- 随时准备好被打断深入，所以每个提过的点你都要能展开。

下一部分 (文档第二部分) :

- 五、五大项目/经历深度讲解 + 面试官可能问的问题 + 参考答案
- 六、Python 高频面试题题库 (从易到难，带代码和讲解)
- 七、SQL 高频面试题题库 (内/外连接、聚合、窗口函数、子查询，从易到难，带答案)
- 八、行为面 STAR 故事 + 想问面试官的问题清单

五、项目 / 经历深度讲解 (面试核心)

面试官深挖项目的通用套路：他们会问「解决了什么问题→为什么用这个技术→有别的方案吗→遇到什么困难→怎么验证效果」。每个项目我都会给你「一句话梗概 (开场) → 详细讲解 (脱稿复述) → 高频提问+参考答案」。

玩法：一个项目背三件事

1. 梗概 (30秒, 开场就用) ——讲清清"What + 用什么 + 带来什么"。
2. 技术脉络 (1分钟) ——能说清选型原因和流程。
3. 深挖应答——下面列的问题至少能答出 2-3 个主流的。

5.1 项目一：stock-monitor (多市场股票监控 + 日报系统)

🗣️ 一句话梗概

"An automated, daily-updated multi-market equity monitoring system that sources quotes and news, analyses them through a configurable multi-provider LLM layer running Buffett/Fisher frameworks with DCF valuation, and serves results on a Flask + ECharts dashboard with Feishu alerts — cutting my manual research time by roughly 90% and surfacing 5+ candidate opportunities a day."

📖 详细讲解 (脱稿用)

- **要解决的问题**：手动盯多个市场、拉数据、读研报、分析估值，耗时且易漏。我想自动化「采集→分析→提醒→可视化」全链路。
- **数据层**：用 `yfinance` 拿股票行情 (多市场)，配了 `AKShare` / `baostock` 作为 fallback (因为其中某些源偶尔会限流/失效，双源保证稳定性)；新闻用 Tavily 搜。
- **分析层**：接了一个 **configurable LLM layer**，能接入多个主流模型供应商 (可切换 provider)，跑两套框架：**巴菲特价值框架** (看自由现金流、ROE、估值安全边际) + **Fisher 成长框架** (看营收/利润增速)，并用 **DCF 估值** 做定量测算，最终输出候选人清单。
- **交付层**：用 **Flask + ECharts** 做交互式网页仪表盘；**飞书(Feishu/Lark)机器人** 实时推送到手机；**GitHub Actions** 每日 cron 自动跑一遍。
- **工程细节**：还做了 cost tracking (每次跑分析记录花了多少 token/成本) 和健康检查 (pipeline 有没有失败，失败会提示)。
- **研发方式**：全程用 Claude Code 做了很多 human-AI pair programming，AI 写代码我 review 结合业务判断。

❓ 面试官可能会问 & 怎么答

Q1: 为什么选多个 LLM 供应商，不固定用一个？

"稳定性 + 成本 + 多样性。不同 provider 在不同时段可能有限流或质量差异，做成可配置的抽象层，出问题能瞬间切换；同时也方便对比效果和成本。核心是把模型当可替换的组件，而不是绑定死一家。"

Q2: Buffett 框架和 Fisher 框架具体怎么用代码实现的？

"两个都是**打分/筛选逻辑**：Buffett 侧重价值——比如计算 ROE 是否高于阈值、自由现金流是否为负、当前 PE/PB 相对历史分位是否合理；Fisher 侧重成长——营收和盈利增速是否持续超 XX%。每个维度有规则权重，最后综合评分，再叠 DCF 算理论价值区间，过滤掉估值过高的。"

Q3: 你的 DCF 估值怎么做？参数从哪来？

"基于自由现金流和增速假设折现。增速来自历史数据和券商一致预期，折现率用 WACC 近似。**关键是敏感性分析**——我会看增速和折现率变动时估值的变化范围，而不是只看单点，避免过度自信。"

Q4: 你说的 90% 节省是怎么衡量的？

"对比口径：以前手动看 20+ 只股票的行情+新闻+研报大约每天要花 1-2 小时；系统跑完一轮全自动，我只需要 5-10 分钟看报表和筛选，所以约 90%。它替代的是**重复的信息搜集**，不是我 100% 的投研判断。"

Q5: 为什么选 Flask 而不是 FastAPI / Django？

"项目规模小、偏内部工具，Flask 轻量够用、上手快。更看重的是把 ECharts 前端和自动化集成好。如果以后要做 API 文档/类型校验或更高并发，会考虑 FastAPI——这其实也是我在项目里评估过的取舍。"

Q6: 遇到的最大困难是什么？（几乎必问！）

"**数据源稳定性**。免费行情源有时限流或改结构，管一天数据抓不到。我的解法是**双源 fallback + 健康检查 + 失败通知**，保证即使一个源挂了系统也能降级运行而不至于 silent fail。这让我学会了把『健壮性』当第一优先的设计原则。"

Q7: 如果让你扩展，你会加什么？

"回测（验证策略历史表现）、更强的风控（止损/仓位）、以及把 LLM 的分析结果加一层可解释的量化校验——避免模型幻觉。也可能加多用户和多市场扩展。"

5.2 项目二：obsidian-vault（全球新闻知识库 + 日报）

一句话梗概

"A GitHub Actions pipeline that aggregates four news sources, uses a DeepSeek-powered router to distill 100+ daily articles into a single 5-minute bilingual briefing, and organizes everything into an Obsidian knowledge base — transforming my daily information overload into a manageable, retained knowledge flow."

📖 详细讲解

- **要解决的问题**：信息过载。每天国内国外、AI 热点、关注的博主、热点资讯太多，看不过来、也记不住。想做成「采集→自动分类路由→精炼摘要→归档可检索」的一条龙。
- **采集层**：GitHub Actions 定时任务聚合 **4 个来源**——AI 热点、我追踪的 X(Twitter) 人物动态、热点话题 feed、AI 从业者/博主的内容。
- **路由核心**：我设计了一个 **DeepSeek-powered 三层漏斗路由(topic router)**——先粗分类(哪个领域)→再判断价值(值不值得进日报)→最后生成摘要+标签。避免所有内容一股脑塞进来，保证**只留高价值、重组**的内容。
- **输出**：自动生成**中英双语**的日报/周报（因为是 bilingual，方便自己读和以后分享），归档成 Obsidian 的 Markdown，可检索、可链接。
- **工程亮点**：零成本监控系统健康（监测任务有没有跑成功、Token 花费控制到接近 0），保证私有使用成本可控。

🔗 面试官可能会问 & 怎么答

****Q1: 三层漏斗路由具体是哪三层？怎么做的？**

"第一层**领域粗分**：这篇文章属于哪个主题（AI、金融、生活.....）；第二层**价值过滤**：对我/对读者有没有增量信息，还是噪声；第三层**精炼生成**：只对通过前两层的文章生成摘要、标签和一句价值判断。好处是把『分类』『过滤』『生成』解耦，各层可以单独调优，也省 token。"

Q2: 为什么用 DeepSeek 不用别的？

"平衡成本和效果。DeepSeek 的 API 性价比高、中文和英文处理都稳，对这种大量内容精炼的场景 token 成本优势明显。也因为我的 LLM 层是可配置的，如果精度要求更高可以临时切更强的模型。"

Q3: 你提到双语，说明你组织多语言内容的能力怎么样？

"我本身中英都流利，项目也要求产出双语内容。这个系统自动把英文源转成中英双语，说明我在多语言内容处理、以及『给不同读者呈现』的用户思维上是顺手且实际锻炼过的。"

Q4: 为什么是 Obsidian？跟 Notion 比呢？

"Obsidian 的核心是本地 **Markdown + 双向链接**，内容存在自己的文件里、不绑定平台、检索速度快，适合长期知识管理。Notion 更偏协作和页面化。我这个更偏『个人知识库 + 强检索』的诉求，所以选 Obsidian。"

Q5: 内容去重怎么做的？

"同一事件不同来源会重复。我用**标题文本相似度 + 路由层的语义判断**去做初步去重，把相似来源并成一条、保留信息最全的。如果以后要更准可以加 embedding 判重。"

5.3 实习经历：Tungee (数据运营实习生)

一句话梗概

"At Tungee I built the data backbone for social content operations — a scheduled pipeline collecting daily KPIs for 50+ KOL accounts on Douyin and Bilibili, Tableau dashboards to visualize it, and data-backed recommendations that roughly doubled the speed at which the operations team could spot top-performing content formats."

详细讲解

- **背景**：Tungee 是 AI 销售情报 SaaS 公司（约1000人），我做的是市场/内容运营的数据支持。
- **Pipeline**：设计并实现了**定时采集**抖音、B站 50+ KOL 账号的核心指标（粉丝、播放、评论），替代之前人工**每天手动记录**——省时且更不容易出错。
- **可视化**：写了**交互式 Tableau 仪表盘**，展示表现趋势和互动模式，团队**每周内容复盘会**都在用。
- **决策价值**：把数据结论喂给内容策略，帮运营团队**更快识别出哪种内容形式表现最好**（约2倍速），直接提升了选题效率。

面试官可能会问 & 怎么答

Q1: 定时采集抖音/B站数据，遇到过反爬吗？

"遇到过，尤其是抖音的签名(a_bogus)类反爬比较强。我的做法是优先合法合规的方式（如官方开放 API / 授权）、做频率控制、必要时用到合法的数据获取路径。**合法 + 有边界是第一原则**——这也让我更关注工程做稳定性而不是硬碰反爬。"

Q2: 50+ 账号、每天跑，技术上是定时任务怎么设计的？

"用 cron 做定时触发，抓取入库（MySQL 这类存储），做成增量更新，失败有重试和通知。核心是**幂等 + 可重跑 + 失败可见**。"

Q3: Tableau 具体看哪些维度？

"粉丝增长趋势、互动率(互动/播放)、不同内容形式的对比、头部 vs 长尾账号分布。核心是帮运营回答『哪种内容值得加大投入』。"

Q4: 你提到的『2x faster』怎么理解？

"更准确的定性表述是：以前要靠人工翻数据凭感觉猜哪种内容形式好，现在 dashboard + 数据能直接在复盘会上快速定位，识别主流表现形式的决策路径明显缩短。我不对它做精确的百分比，但确实是可感知的效率提升。"

Q5: 团队协作中最难的地方？

"业务和数据的对齐——运营同事要的是『下一条该发什么』，但数据只能给『过去什么表现好』。我的办法是把报表做成**可采取行动的**(actionable)——每张图都导向一个可操作的结论，而不是只丢一堆数字。"

5.4 研究经历：广外东南亚新闻采集项目

可做为「团队科研 + 爬虫 + 数据管理」能力的补充，面试不一定会问，但可作为行为面素材。

一句话：在科研项目中负责东南亚多源新闻数据的采集管道（泰国/印尼/新加坡的旅游文化、政府站点），用定时增量爬虫(requests+MySQL)持续供给标注流水线，并做了系统稳定性和覆盖率的升级。

可能的深挖：为什么选这几个来源、增量爬虫怎么避免重复采集（维护已抓的指纹/去重）、怎么处理多语言/编码、MySQL 表结构怎么设计。

六、Python 高频面试题库（从易到难）

每题先给「考点」，再给「代码」，再给「一句话讲解」。准备方式：先自己手写一遍，再看答案。

6.1 基础（必会）

Q1. 反转一个字符串（不能用内置）

```
s = "hello"
print(s[::-1])          # 'olleh' — 切片反转，最简单
# 面试可能是写循环版本：
def rev(s):
    r = ""
    for ch in s:
        r = ch + r
    return r
```

Q2. 判断一个数是偶数还是奇数

```
n = 7
print("even" if n % 2 == 0 else "odd")
```

Q3. 求一个列表里最大/最小/总和

```
lst = [3, 1, 4, 1, 5, 9, 2, 6]
print(max(lst), min(lst), sum(lst))
```

Q4. 去重并保持顺序

```
lst = [1,3,1,2,3,4]
seen = []
for x in lst:
    if x not in seen:
        seen.append(x)
print(seen) # [1,3,2,4]
# 或用 dict.fromkeys
print(list(dict.fromkeys(lst)))
```

Q5. 统计一个字符串里每个字母出现的次数

```
from collections import Counter
print(Counter("hello world"))
# => Counter({'l':3, 'o':2, 'h':1, ...})
```

6.2 数据结构 (字典/list 操作)

Q6. 合并两个字典 (Python 3.9+)

```
d1={"a":1}; d2={"b":2}
merged = d1 | d2
merged.update(d2) # 旧写法: d1.update(d2)
```

Q7. 按 value 给字典排序 (高频)

```
d={"a":3,"b":1,"c":2}
sorted(d.items(), key=lambda kv: kv[1]) # 升序
sorted(d.items(), key=lambda kv: kv[1], reverse=True) # 降序
```

Q8. 找出列表中出现次数最多的元素

```
from collections import Counter
lst=[1,2,2,3,2,1]
print(Counter(lst).most_common(1)) # [(2, 3)]
```

Q9. list、tuple、set、dict 的区别 (口头题)

list: 有序可变; tuple: 有序不可变; set: 无序不重复; dict: 键值对。适用的场景和是否可哈希。

6.3 函数与装饰器

Q10. 写一个装饰器计算函数耗时 (高频)

```
import time

def timer(fn):
    def wrapper(*a, **kw):
        t0=time.time()
        res=fn(*a, **kw)
        print(f"{fn.__name__} took {time.time()-t0:.4f}s")
        return res
    return wrapper

@timer
def work():
    time.sleep(0.1)

work()
```

Q11. lambda + map/filter/reduce

```
nums=[1,2,3,4,5]

squares=list(map(lambda x:x*x, nums))

evens=list(filter(lambda x:x%2==0, nums))

from functools import reduce

total=reduce(lambda a,b:a+b, nums)
```

6.4 面向对象 / 常见内置

Q12. 什么是 `__init__`、`self` ? (口头)

构造方法初始化实例，`self` 指向当前实例。

Q13. 浅拷贝 vs 深拷贝

```
import copy

a=[[1,2],[3,4]]

b=copy.copy(a)    # 浅：外层新对象，内层 list 仍是同一引用

c=copy.deepcopy(a) # 深：完全独立副本
```

6.5 算法/逻辑 (中高难度 , Data 岗常见)

Q14. 判断两个字符串是不是互为回文排列/anagram

```
from collections import Counter

def is_anagram(s,t):
    return Counter(s)==Counter(t)
```

Q15. 求数组中两数之和等于 target (高频)

```
def two_sum(nums,target):
    seen={}
    for i,v in enumerate(nums):
        if target-v in seen:
            return [seen[target-v],i]
        seen[v]=i
    return None

print(two_sum([2,7,11,15],9)) # [0,1]
```

Q16. 给列表去重并统计频率，找 top K (Data 岗常考)

```
from collections import Counter

def top_k(nums,k):
    return [x for x,_ in Counter(nums).most_common(k)]
```

Q17. 二分查找 (要会写)

```
def bsearch(arr,target):
    lo,hi=0,len(arr)-1
    while lo<=hi:
        mid=(lo+hi)//2
        if arr[mid]==target: return mid
        elif arr[mid]<target: lo=mid+1
        else: hi=mid-1
    return -1
```

6.6 Pandas 专项 (Data 岗非常高频)

Q18. 读 CSV、看前几行、基本信息

```
import pandas as pd
df=pd.read_csv("data.csv")
df.head()          # 前5行
df.info()          # 列、类型、非空个数
df.describe()      # 数值列统计
```

Q19. 筛选 (布尔索引)

```
df[df["age"]>30]          # 年龄>30
df[(df["age"]>30)&(df["score"]<80)]
df[df["city"].isin(["London","Birmingham"])]
```

Q20. 分组聚合

```
df.groupby("city")["sales"].mean()    # 各城市销售均值
df.groupby("city").agg({"sales":"sum","orders":"count"})
```

Q21. 处理缺失值

```
df.isnull().sum()          # 看每列缺失
df["col"].fillna(df["col"].mean())    # 用均值填
df.dropna()                # 删含缺失的行
```

Q22. 新增列 / 排序

```
df["revenue"]=df["price"]*df["qty"]
df.sort_values("revenue",ascending=False).head(10)
```

下一部分 : **SQL 高频题库 (从易到难 , 重点 !)** + 行为面 + 反问清单。

七、SQL 高频面试题库 (重点 ! 从易到难)

SQL 是 Data 岗面试出现频率最高的技术题。你要把「连接JOIN」「聚合GROUP BY」「窗口函数 Window」「子查询」这四大块吃透。建议 : 先看「核心概念一句话」→ 自己写一遍案例 → 再对答案。

7.0 🗝️ SQL 核心概念速记 (背这几点就不慌)

概念	一句话	语法关键词
查全表	从一个表取所有行所有列	<code>SELECT * FROM t;</code>
列筛选	只取需要的列	<code>SELECT name, age FROM t;</code>
去重	去掉重复行	<code>SELECT DISTINCT city FROM t;</code>
筛选行	条件过滤	<code>WHERE</code>
排序	排序	<code>ORDER BY col ASC/DESC</code>
分组	按某列分组做统计	<code>GROUP BY</code>
分组后过滤	对分组结果再过滤	<code>HAVING</code> (因为WHERE不能用在聚合后)
内连接	两个表都匹配到的行	<code>INNER JOIN</code> / <code>JOIN</code>
左连接	左表全保留，右表匹配到才补	<code>LEFT JOIN</code>
右连接	右表全保留	<code>RIGHT JOIN</code>
全连接	两边都保留	<code>FULL OUTER JOIN</code>
窗口函数	在"每个分组内"做计算，不合并行	<code>OVER(PARTITION BY ...)</code>
子查询	把一个查询的结果当另一个查询的输入	<code>WHERE x IN (SELECT ...)</code>

⚠️ SQL 执行顺序 (面试常考，非常爱问！)

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

记住：WHERE 在 GROUP BY 之前执行，所以 WHERE 里不能写聚合条件，要用 HAVING。

7.1 我们先造两张测试表（自己练习用）

-- 员工表

```
CREATE TABLE employees (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    dept VARCHAR(50),  
    salary INT,  
    hire_date DATE  
);
```

-- 部门表

```
CREATE TABLE departments (  
    dept VARCHAR(50) PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

-- 数据

```
INSERT INTO employees VALUES  
(1, 'Alice', 'eng', 90000, '2021-01-10'),  
(2, 'Bob', 'eng', 85000, '2020-06-01'),  
(3, 'Carol', 'sales', 70000, '2022-03-15'),  
(4, 'Dave', 'eng', 95000, '2019-09-20'),  
(5, 'Eve', 'sales', 65000, '2023-01-05'),  
(6, 'Frank', 'hr', 60000, '2020-11-11'),  
(7, 'Grace', NULL, 75000, '2021-04-25'); -- 注意Grace部门是NULL
```

```
INSERT INTO departments VALUES  
( 'eng', 'Engineering'),  
( 'sales', 'Sales & Marketing'),  
( 'hr', 'Human Resources'),  
( 'fin', 'Finance'); -- 注意fin没有员工
```

7.2 入门 (必会 , 热手)

Q1. 查所有员工的姓名和薪水 , 按薪水从高到低

```
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

Q2. 查部门为 eng 的员工名字 (去重部门)

```
SELECT name FROM employees WHERE dept='eng';
SELECT DISTINCT dept FROM employees;    -- 看有哪些部门
```

Q3. 统计一共有多少员工、总薪水、平均薪水

```
SELECT COUNT(*) AS emp_count,
       SUM(salary) AS total_salary,
       AVG(salary) AS avg_salary
FROM employees;
```

`COUNT(*)` 数所有行 ; `COUNT(col)` 只数非空。

Q4. 薪水在 70000 到 90000 之间的员工

```
SELECT name, salary FROM employees
WHERE salary BETWEEN 70000 AND 90000;
-- 等价: WHERE salary>=70000 AND salary<=90000
```

Q5. 「每个部门」的平均薪水、最高薪水、人数

```
SELECT dept,
       AVG(salary) AS avg_sal,
       MAX(salary) AS max_sal,
       COUNT(*) AS emp_cnt
FROM employees
GROUP BY dept;
```

GROUP BY 考点 : select 里出现的非聚合列 , 必须出现在 GROUP BY 里。

7.3 JOIN 连接 (重点中的重点 🔥)

Q6. 内连接(INNER JOIN) : 员工 + 他的部门名称

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept = d.dept;
```

结果：只有匹配到部门的员工 (Grace 的 dept 是 NULL，匹配不上，所以不会出现；fin 部门没员工也没有)。记住：INNER JOIN 只保留两边都能匹配上的行。

Q7. 左连接(LEFT JOIN) : 所有员工，没部门也显示 (显示NULL)

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept = d.dept;
```

结果：所有员工都会出现，Grace 匹配不到部门，部门名显示 NULL。这才是"保留左表全部"。面试爱问：INNER 和 LEFT 区别 → INNER只留匹配，LEFT保留左表所有+NULL。

Q8. 右连接(RIGHT JOIN) : 所有部门，就算没员工也出现

```
SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept = d.dept;
```

结果：所有部门都出现，fin 部门没有员工，name 显示 NULL。

Q9. 全外连接(FULL OUTER JOIN) : 两边都保留

```
SELECT e.name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON e.dept = d.dept;
```

结果：所有员工 + 所有部门都出现，没匹配到的补 NULL。(注意：MySQL 标准 SQL 不支持 FULL OUTER，用 UNION 模拟，见Q20。)

Q10. 找出「没有对应部门」的员工 (左连接 + IS NULL)

```
SELECT e.name
      FROM employees e
     LEFT JOIN departments d ON e.dept = d.dept
    WHERE d.dept IS NULL;
```

非常经典！ 结果：Grace (dept=NULL)。这个"排除掉匹配到的"技巧面试超爱考。

7.4 聚合 + HAVING

Q11. 找出「平均薪水 > 70000」的部门

```
SELECT dept, AVG(salary) AS avg_sal
      FROM employees
     GROUP BY dept
    HAVING AVG(salary) > 70000;
```

关键：因为是对"聚合结果"过滤，不能用 *WHERE* (*WHERE* 在 *GROUP BY* 之前执行)，要用 **HAVING**。

Q12. 哪些部门人数大于1

```
SELECT dept, COUNT(*) c
      FROM employees
     GROUP BY dept
    HAVING COUNT(*) > 1;
```

eng:2(sales有2人...实际Alice/Bob/Dave是eng=3人)，结果：eng(3), sales(2)。

7.5 窗口函数 Window Function (进阶🔥，Data岗高概率考)

核心区别： `GROUP BY` 会把多行合并成一行；窗口函数不会——它保留所有行，同时在每行上算一个基于分组的值。这是它和 `GROUP BY` 最大的考点。

语法：

```
SELECT col,  
       ROW_NUMBER() OVER (PARTITION BY col2 ORDER BY col3) AS rn,  
       RANK()          OVER (...),  
       DENSE_RANK()    OVER (...),  
       SUM(col)         OVER (PARTITION BY col2 ORDER BY col3) AS running_total  
FROM t;
```

三个排名函数的区别（必背！）

```
SELECT name, dept, salary,  
       ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) AS rn,  
       RANK()          OVER (PARTITION BY dept ORDER BY salary DESC) AS rk,  
       DENSE_RANK()    OVER (PARTITION BY dept ORDER BY salary DESC) AS dr  
FROM employees;
```

name	dept	salary	rn	rk	dr
Dave	eng	95000	1	1	1
Alice	eng	90000	2	2	2
Bob	eng	85000	3	3	3
Carol	sales	70000	1	1	1
Eve	sales	65000	2	2	2

区别记忆：ROWNUMBER 无重复编号(1,2,3...)；RANK 相同值并列、会跳号(1,1,3)；DENSERANK 并列不跳号(1,1,2)。

面试最经典的窗口函数题——Top N per group：

Q13. 找出「每个部门工资最高的那个人」

```
SELECT name, dept, salary  
FROM (  
    SELECT name, dept, salary,  
           ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) AS rn  
    FROM employees  
    ) t  
WHERE t.rn = 1;
```

这是全英最具代表性的 Data 面试题之一。思路：先用窗口函数在每组内排名，再在外层筛第1名。
⚠ 注意：子查询里的窗口函数结果要用外层 WHERE 过滤，不能直接 WHERE rn=1(窗口函数在 SELECT 阶段才计算)。

Q14. 累计求和(running total)：按工资累积

```
SELECT name,  
       salary,  
       SUM(salary) OVER (ORDER BY salary) AS running_total  
FROM employees;
```

ORDER BY 在 OVER 内 = 从这个分组开头累积到当前行，做 running total。

Q15. 窗口函数 vs GROUP BY 的区别（口头题，常考）

"GROUP BY 把多个行折叠成一个聚合行；窗口函数在不改变行数的情况下，给每一行附加一个聚合计算结果。一个想只看总数用 GROUP BY；一个既想看明细又想看聚合(如占比、排名、累计)用窗口函数。"

7.6 子查询 Subquery

Q16. 找工资高于全公司平均的员工

```
SELECT name, salary  
FROM employees  
WHERE salary > (SELECT AVG(salary) FROM employees);
```

子查询先算出平均，外层用它比较。

Q17. 找「那些部门的平均工资比我高」的部门 (相关子查询,较进阶)

```
SELECT DISTINCT e.dept
FROM employees e
WHERE e.dept IN (
    SELECT dept FROM employees
    GROUP BY dept
    HAVING AVG(salary) > 78000
);
```

Q18. EXISTS 用法：找出「有员工的部门」(变相实现 JOIN 判断)

```
SELECT d.dept
FROM departments d
WHERE EXISTS (
    SELECT 1 FROM employees e WHERE e.dept = d.dept
);
```

EXISTS 判断"是否存在至少一行匹配"，比 IN 在大表时常更高效。结果：eng, sales, hr (没有 fin)。

7.7 综合 / 进阶

Q19. 经典的「第二高薪」(含去重的进阶题)

```
-- 找全公司第二高的薪水
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

或用窗口：`SELECT salary FROM (SELECT salary, DENSE_RANK() OVER(ORDER BY salary DESC) rk FROM employees) t WHERE rk=2 LIMIT 1;`

Q20. 用 UNION 模拟 FULL OUTER JOIN (MySQL不支持 FULL)

```
SELECT name, dept FROM employees
LEFT JOIN departments USING(dept)
UNION
SELECT name, dept FROM employees
RIGHT JOIN departments USING(dept);
```

左右连接的结果合并去重，模拟全连接。

Q21. 日期时间常见题

```
-- 2024年的员工
SELECT * FROM employees WHERE hire_date >= '2024-01-01' AND hire_date < '2025-01-01';
-- 或：WHERE YEAR(hire_date)=2024
-- 入职年数
SELECT name, DATEDIFF(CURDATE(), hire_date)/365 AS years FROM employees;
```

7.8 面试官可能考的 SQL 场景题 (DATA岗常见)

场景A：「给你一张用户表(users: id, signupdate) 和订单表(orders: userid, amount, order_date)，统计每个月的注册用户数和订单总额」

```
-- 每月注册数
SELECT DATE_FORMAT(signup_date,'%Y-%m') AS month, COUNT(*)
FROM users GROUP BY month;
-- 每月订单总额
SELECT DATE_FORMAT(order_date,'%Y-%m') AS month, SUM(amount)
FROM orders GROUP BY month;
-- 每月每用户平均消费
SELECT DATE_FORMAT(order_date,'%Y-%m') AS month, SUM(amount)/COUNT(DISTINCT user_id)
FROM orders GROUP BY month;
```

场景B：「找出连续登录3天以上的用户」(进阶高频)


```
WITH t AS (  
    SELECT user_id, login_date,  
           -- 用登录日 - 行号 判断连续性:连续登录的 这个差值相同  
           DATE_SUB(login_date, INTERVAL ROW_NUMBER() OVER(PARTITION BY user_id ORDER B  
Y login_date) DAY) AS grp  
    FROM login_log  
)  
  
SELECT user_id  
FROM t  
GROUP BY user_id, grp  
HAVING COUNT(*) >= 3;
```

CTE(**WITH ... AS**) 是进阶加分项，能写出来很加分。逻辑：连续日期减去行号得到相同的分组键。

7.9 SQL 备战建议（针对你“忘光了、没刷题”的现状）

1. **先动手跑一遍**：本机装 MySQL/SQLite 或用 sqlite3，把7.1的建表语句跑起来，逐条试上面的题，亲眼看到结果。
2. **死磕四大块**：JOIN(内外)、GROUP BY+HAVING、窗口函数(RANK系+SUM OVER)、子查询。会了这四样，90%的SQL题能扛。
3. **背诵执行顺序**：FROM→WHERE→GROUP BY→HAVING→SELECT→ORDER BY。面试问到必答。
4. **每道题动手写一遍**，别只看：SQL没有比手写更快的提升方式。
5. 有一题不会别慌，面试官常给提示，说思路比沉默强。

八、行为面 STAR + 反问清单

8.1 STAR 法则（讲故事的结构）

- **Situation** 情境：背景是什么
- **Task** 任务：你要解决什么
- **Action** 行动：你具体做了什么（强调你的贡献）
- **RResult** 结果：带来了什么（能量化就量化）

8.2 把常见行为问题套进你的真实经历

面试官问	用哪个经历套
讲一个你解决复杂问题的例子	stock-monitor (数据源失败→双源fallback+健康检查)
讲一个你主动承担/带结果的例子	obsidian-vault (从0搭完整系统)
讲一个团队协作/跨部门的例子	Tungee (跟运营团队对齐"动作型报表")
讲一个你克服困难的例子	毕业论文 (生成式vs判别式对比，踩坑→优化)
讲一个你学得快/自学的例子	从统计学出身快速上手爬虫、LLM、前端dashboard
为什么想做数据/AI	数学统计背景 + 真正动手做系统体会到数据带来决策的价值

8.3 结尾一定要反问 (展现兴趣和成熟度)

好的反问 (选几个) :

- "这个岗位前3个月最重要的目标是什么?"
- "团队现在最想解决的 data 问题是什么?"
- "团队的技术栈和日常工具链是怎样的?"
- "对这个岗位，您觉得什么样的人能做得最出色?"
- "新人的培养和成长路径是怎样的?"

避免问：薪资多少(还没到时候)、福利、几号放假。

九、面试当天 checklist (临场冲刺)

- ☐ 提前 10 分钟到场 / 设备、网络、摄像头测试好
- ☐ 自我介绍精简版脱口而出
- ☐ 两个项目的梗概 + 技术脉络 + 2-3个深挖题答案在脑中
- ☐ SQL四大块 + Python高频题过一遍
- ☐ 准备 2-3 个反问
- ☐ 带好简历打印件 + 项目 GitHub 链接可展示
- ☐ 心态：面试是双向选择，你在评估公司是否适合你

🔥 后续我可以帮你：

1. 对某个具体 Job Description(J.D.) 定制关键词，调整自我介绍和项目表述
2. 做一个"面试模拟"，模拟面试官随机抽题考你，你答我点评
3. 写一封可复用的 Cover Letter 模板
4. 按你投递的具体岗位准备冷门/进阶题

